

UNIVERSIDAD DE LOS ANDES
Facultad de Ingeniería y Ciencias Aplicadas
Arquitectura de Computadores 2026-20

Proyecto N° 3

Del RTL al GDSII: el Pochoco SoC en GF180MCU con LibreLane

Grupo 16:
Andrés Berríos
Víctor Parada
Sebastián Trigo

Profesor de Cátedra: Jorge Gómez M.

3 de noviembre de 2026

Resumen

Este informe documenta la implementación del Pochoco SoC, con el juego de reflejos del Proyecto 2, como un macro digital de celdas estándar en la tecnología GF180MCU, desde el RTL hasta el GDSII, usando el flujo abierto LibreLane. Primero se identificaron y reemplazaron las construcciones del RTL que dependían de la FPGA iCE40: la BRAM del banco de registros, la carga de la memoria con `$readmemh`, el reset de encendido con valor inicial, la frecuencia de reloj asumida y los puertos propios de la placa. Para la memoria del programa se compararon dos estrategias generadas automáticamente desde `game.s`: una ROM combinacional con una RAM de datos reducida, y una memoria unificada de Flip-Flops.

La primera resultó 4,2 veces más pequeña en área de celdas y se eligió como arquitectura definitiva, con una RAM de 2 palabras, ya que el juego no usa memoria de datos. El diseño final ocupa 151.424,9 μm^2 de celdas en un *die* de 0,407 mm^2, tiene 909 Flip-Flops y cierra *timing* hasta 53,2 ns (18,80 MHz). Con $T = 100$ ns y $U = 40\,\%$ termina con DRC y LVS limpios, sin violaciones de antena y cumpliendo setup y hold en todos los corners. La simulación del netlist posterior a la síntesis reproduce, ciclo a ciclo, el comportamiento del RTL.

El código fuente del proyecto (RTL del macro, `assembler_v2.py`, archivos de configuración de LibreLane, testbench de simulación) y los archivos de evidencia de cada sección se encuentran en el repositorio https://github.com/ViccioPro/proyecto_3-arquitectura_computadores.git.

Índice

1. Introducción **4**

2. Entorno de trabajo y reproducibilidad **5**

3. Del RTL de FPGA al RTL de ASIC **6**

3.1. Módulo superior `asic_top.v` 6

3.2. Construcciones dependientes de la FPGA 6

3.2.1. BRAM `SB_RAM40_4K` en el banco de registros 6

3.2.2. Inicialización de la memoria con `$readmemh` 7

3.2.3. Reset de encendido con valor inicial (`por_cnt`) 7

3.2.4. Frecuencia de reloj asumida (`TENTH_CYCLES`) 7

3.2.5. Puertos propios de la placa 7

4. Memoria, programa del juego y extensión del assembler **9**

4.1. Uso real de memoria 9

4.2. Extensión del assembler (`assembler_v2.py`) 9

4.3. Estrategias de memoria implementadas 9

4.3.1. Estrategia 1: ROM combinacional y RAM de datos reducida 10

4.3.2. Estrategia 2: memoria unificada de Flip-Flops con reset y set 10

4.4. Comparación entre estrategias 10

5. Síntesis y trazabilidad jerárquica **12**

5.1. Método de trazabilidad 12

5.2. Área por módulo 13

5.3. Celdas más usadas 14

5.4. Comparación con lo esperado 15

6. Timing: barrido de período, ruta crítica y constantes del juego **16**

6.1. Barrido de `CLOCK_PERIOD` 16

6.2. Frecuencia máxima 17

6.3. Ruta crítica 17

6.4. Modificación del RTL 19

6.4.1. Primer intento: señal `pc_set_o` del controlador 19

6.4.2. Cambio propuesto: lectura de periféricos sin habilitación 19

6.5. Constantes del juego para el período T 21

7. Floorplan, red de alimentación y placement **22**

7.1. Barrido de utilización del core 22
7.2. Red de alimentación 24
7.3. Terminales del macro 25
7.4. Costo estimado 26

8. Signoff y layout **27**
8.1. Resultados de DRC, LVS y antena 27
8.2. Violación de antena detectada y corregida 27
8.3. Violaciones de *max slew* y *max cap* 28

9. Simulación funcional post-síntesis **32**
9.1. Configuración 32
9.2. Testbench 33
9.3. Resultados 33
9.4. Archivos entregados 34

10. Conclusión del Proyecto **35**

1. Introducción

En el Proyecto 2 se desarrolló un juego de reflejos que se ejecuta sobre el Pochoco SoC, un sistema en chip basado en el procesador RISC-V Espino, implementado en la FPGA iCE40 de la tarjeta Go Board. En una FPGA, el hardware se configura sobre recursos ya fabricados: bloques de memoria, inicialización de registros al cargar la configuración y pines conectados directamente a la placa. Un ASIC, en cambio, se fabrica a partir de celdas estándar, y cada uno de esos recursos debe resolverse explícitamente en el diseño.

El objetivo de este proyecto es llevar ese mismo SoC, con el programa del juego, desde el RTL hasta un GDSII en la tecnología GF180MCU, usando el flujo abierto LibreLane. El resultado es un macro digital implementado con celdas estándar, no un chip completo listo para encapsular: sus puertos son terminales lógicos que, en un chip completo, se conectarían a *pads* de I/O con protección y manejo de carga.

Todas las corridas finales usan los parámetros comunes a todos los grupos: un período de reloj objetivo $T = 100$ ns (10 MHz) y una utilización objetivo del core $U = 40\,\%$. Una corrida se considera cerrada cuando LibreLane completa el flujo, el diseño tiene DRC y LVS limpios, no presenta violaciones de antena sin justificar y cumple setup y hold, con WNS ≥ 0 y TNS $= 0$, en los corners exigidos.

El informe se organiza siguiendo las actividades del proyecto. La Sección 2 registra las herramientas y versiones utilizadas. La Sección 3 identifica las construcciones del RTL que dependían de la FPGA y cómo se reemplazaron. La Sección 4 analiza el uso real de memoria del juego y compara las dos estrategias de memoria implementadas. La Sección 5 analiza la síntesis y la trazabilidad de los bloques del diseño. La Sección 6 determina la frecuencia máxima, identifica la ruta crítica, evalúa una modificación del RTL y calcula las constantes del juego. La Sección 7 explora el floorplan, la red de alimentación, los terminales del macro y el costo estimado. La Sección 8 presenta las verificaciones finales y la ubicación de los bloques en el layout, y la Sección 9 verifica el netlist mediante simulación. Finalmente, la Sección 10 resume los resultados y las lecciones del proyecto.

2. Entorno de trabajo y reproducibilidad

El diseño se llevó desde el RTL del Proyecto 2 hasta un GDSII con el flujo abierto LibreLane y el PDK GF180MCU. La Tabla 1 registra las versiones utilizadas en la corrida final, que también deben quedar en el README del repositorio.

Elemento	**Versión**
LibreLane	3.0.17 (instalado con Nix)
PDK	`gf180mcuD`, revisión de Open PDKs `54435919abffb937387ec956209f9cf5fd2dfbee` (gestionada con Ciel)
Biblioteca de celdas estándar	`gf180mcu_fd_sc_mcu7t5v0`
Yosys / Verilator	0.62 / 5.044 (incluidos en el entorno de LibreLane)

Tabla 1: Versiones de las herramientas, el PDK y la biblioteca de celdas.
Fuente: Elaboración Propia.

Todas las corridas usan `CLOCK_PERIOD` = 100 ns y `FP_CORE_UTIL` = 40 %, salvo los barridos de las Secciones 6 y 7, que modifican solo uno de esos parámetros mediante archivos de configuración separados (`config_timing_*.json` y `config_util_*.json`).

3. Del RTL de FPGA al RTL de ASIC

En esta sección se describen las modificaciones realizadas sobre el Pochoco SoC y el juego de reflejos del Proyecto 2 para llevar el diseño desde la FPGA iCE40 hacia el PDK GF180MCU mediante LibreLane.

3.1. Módulo superior `asic_top.v`

Debido a las diferencias entre la tarjeta Go Board y el flujo ASIC, el módulo superior `game_top.v` se reemplazó por `asic_top.v`, que define la interfaz lógica del macro digital.

En la FPGA, los puertos se asociaban a pines físicos de la placa mediante `goboard.pcf`. En `asic_top.v`, en cambio, los puertos son solo terminales lógicos del macro: no incluyen *buffers* de salida ni protección ESD, que en un chip completo estarían en los *pads* de I/O.

La interfaz expone el reloj (`clk_i`), un reset externo activo en bajo (`rst_ni`) que reemplaza al reset de encendido implícito, los botones (`btn_i[3:0]`), los LEDs (`led_o[3:0]`), los dos displays de siete segmentos (`seg1_o[6:0]` y `seg2_o[6:0]`, con el bit 0 para el segmento A y el bit 6 para el G) y las cuatro señales del bus SPI (`spi_sclk_i`, `spi_mosi_i`, `spi_cs_ni` y `spi_miso_o`). Se eliminaron los puertos UART (`o_UART_TX` e `i_UART_RX`), que solo existían por el pinout de la placa y que el juego no utiliza.

3.2. Construcciones dependientes de la FPGA

Para identificar las construcciones del RTL del Proyecto 2 incompatibles con un ASIC, se ejecutó LibreLane sobre el RTL original y se corrigió una construcción a la vez, volviendo a correr el flujo después de cada cambio para obtener la evidencia de la siguiente. Los registros correspondientes están en la carpeta `evidencia_3_1` del repositorio. A continuación se describe cada construcción, el mensaje del flujo que la evidenció y cómo se reemplazó.

3.2.1. BRAM `SB_RAM40_4K` en el banco de registros

Con el RTL original, el flujo se detuvo en la etapa de *lint* con el error `%Error-MODMISSING: Cannot find file containing module: 'SB_RAM40_4K'` (`bram_modmissing.log`, líneas 1 y 13), provocado por las dos instancias de esa primitiva en `espino_register_file.v` (líneas 22 y 33). `SB_RAM40_4K` es un bloque de memoria exclusivo de las FPGA iCE40 de Lattice y no existe en GF180MCU.

El banco de registros se reescribió con Flip-Flops: un arreglo de 15×32 bits para `x1` a `x15`, con dos puertos de lectura. Se conservó la lectura sincrónica de un ciclo de la BRAM original, de la que depende el *pipeline* de Espino, y se forzó `x0` a cero, ya que en la FPGA ese valor provenía de la inicialización de la BRAM (`INIT_0 = 0`) y en silicio los Flip-Flops no se inicializan.

3.2.2. Inicialización de la memoria con `$readmemh`

Con el banco de registros corregido, el *lint* no emitió ninguna advertencia sobre `$readmemh` en `pochoco_ram.v` (línea 36), y Yosys solo registró la nota informativa `memory init port` para `u_pochoco.u_ram.mem` (`yosys_readmemh.log`, línea 988), con la que cargó `game.hex` como contenido inicial de la memoria. Es decir, el flujo aceptó la construcción sin advertirla como problema.

Sin embargo, en un macro de celdas estándar no existe ningún mecanismo que cargue el contenido de un archivo al energizar el circuito: la memoria implementada con Flip-Flops arrancaría con valores indeterminados y sin el programa del juego. Por eso se eliminó `$readmemh` del RTL, y la carga del programa se resolvió con las dos estrategias de la Sección 4.

3.2.3. Reset de encendido con valor inicial (`por_cnt`)

Verilator emitió la advertencia `PROCASSINIT` (`lint_sin_bram.log`, línea 13) por la declaración `reg [15:0] por_cnt = 16'b0` en `pochoco_soc.v` (línea 40), y Yosys registró el valor inicial en la etapa `PROC_INIT` (`yosys_readmemh.log`, líneas 287 a 289). En la FPGA ese valor lo carga la configuración del iCE40, pero las celdas de GF180MCU no tienen valor inicial, por lo que en silicio el contador arrancaría en un valor desconocido y el reset podría no generarse correctamente.

Se eliminó el contador y se agregó la entrada externa `rst_ni` en `asic_top.v`. En `pochoco_soc.v` esa entrada pasa por un sincronizador de dos Flip-Flops, que activa el reset de forma asíncrona y lo libera de forma sincrónica con el reloj, para evitar metaestabilidad al soltarlo.

3.2.4. Frecuencia de reloj asumida (`TENTH_CYCLES`)

El *lint* registró la advertencia `WIDTHEXPAND` en `pochoco_periph.v`, línea 46 (`lint_sin_bram.log`, línea 1). La advertencia en sí solo indica que el divisor `tenth_divider`, de 22 bits, se compara con un parámetro de 32 bits, pero revela que el divisor de décimas de segundo está dimensionado para `TENTH_CYCLES` = 2.500.000, es decir, para el reloj de 25 MHz de la Go Board. El flujo no advierte este supuesto: se identificó al revisar el parámetro. Para el período objetivo de 100 ns (10 MHz), una décima de segundo equivale a 1.000.000 de ciclos, valor que se fijó en `asic_top.v` (`TenthCycles = 1000000`). El cálculo se detalla en la Sección 6.5.

3.2.5. Puertos propios de la placa

El *lint* registró la advertencia `UNUSEDSIGNAL` para la entrada `i_UART_RX` de `game_top.v`, línea 5 (`lint_sin_bram.log`, línea 7). Este puerto y `o_UART_TX` (fijo en alto) solo existían porque sus pines aparecían en `goboard.pcf`. Ambos se eliminaron, y `asic_top.v` quedó solo con los puertos del juego y del bus SPI.

La Tabla 2 resume las sustituciones. Todos los archivos de registro mencionados están en `evidencia_3_1`.

Construcción FPGA	**Evidencia (archivo, línea)**	**Problema en silicio**	**Reemplazo**
Instancias de `SB_RAM40_4K`	`bram_modmissing.log` (L1, L13): `MODMISSING`	Primitiva exclusiva de iCE40, inexistente en GF180MCU	Banco de 15×32 Flip-Flops con lectura sincrónica y `x0` forzado a 0.
`$readmemh` en `pochoco_ram.v`	`yosys_readmemh.log` (L988): `memory init port` (solo nota informativa)	No hay mecanismo para cargar un `.hex` al encender	Directiva eliminada; carga del programa según la Sección 4.
Reset POR (`por_cnt`)	`lint_sin_bram.log` (L13): `PROCASSINIT`; `yosys_readmemh.log` (L287–289): `PROC_INIT`	Flip-Flops sin valor inicial en silicio	Entrada `rst_ni` en `asic_top` y sincronizador de 2 FF en `pochoco_soc`.
Reloj asumido de 25 MHz	`lint_sin_bram.log` (L1): `WIDTHEXPAND` (indirecta)	Décimas de segundo incorrectas a 10 MHz	`TenthCycles = 1000000` en `asic_top.v`.
Puertos UART de la placa	`lint_sin_bram.log` (L7): `UNUSEDSIGNAL`	Puertos sin función en el macro	Eliminados; `asic_top.v` solo con puertos del juego y SPI.

Tabla 2: Resumen de construcciones dependientes de la FPGA y su reemplazo.
Fuente: Elaboración Propia.

4. Memoria, programa del juego y extensión del assembler

Como en silicio no existe un mecanismo para inicializar memorias desde un archivo, se analizó el uso real de memoria del juego y se implementaron y compararon dos estrategias para almacenar sus instrucciones y datos.

4.1. Uso real de memoria

El consumo de memoria del programa se obtuvo con dos análisis:

1. **Instrucciones:** el comando `grep -c . sw/game.hex` indica que el programa tiene 100 palabras de 32 bits. Para asegurar que ese archivo corresponde al programa actual, se regeneró desde `game.s` con el assembler y se comparó con `diff`: ambos son idénticos.
2. **Datos:** todas las instrucciones de carga y almacenamiento de `game.s` usan como base el registro `x2`, y la única instrucción que escribe `x2` es `lui x2, 0x80000` (dirección `0x80000000`). En `pochoco_soc.v`, un acceso con el bit 31 de la dirección en 1 activa `is_mmio`, por lo que todos los accesos van a los periféricos o al bus SPI y ninguno llega a la RAM. El juego no tiene variables en memoria ni usa *stack*: requiere 0 palabras de datos.

Ambos valores están por debajo de los límites recomendados de 256 palabras de instrucciones y 64 de datos.

4.2. Extensión del assembler (`assembler_v2.py`)

Para generar el hardware de memoria a partir del software, se desarrolló `assembler_v2.py`, que extiende el assembler del Proyecto 2 (`assembler_v1.py`). A partir de `game.s`, genera `game.hex` (idéntico al del Proyecto 2) y el módulo Verilog de cada estrategia de memoria (Sección 4.3). Su uso se muestra en el Listado 1.

Listado 1: Uso de assembler_v2.py. Fuente: Elaboración Propia.

```
# Estrategia 1: game.hex y ROM combinacional
python3 assembler_v2.py sw/game.s sw/game.hex rtl/mem_s1/pochoco_rom.v

# Estrategia 2: ademas, memoria unificada de Flip-Flops
python3 assembler_v2.py sw/game.s sw/game.hex rtl/mem_s1/pochoco_rom.v \
       rtl/mem_s2/pochoco_ram.v
```

4.3. Estrategias de memoria implementadas

Ambas estrategias tienen la misma interfaz que la memoria original (un puerto de instrucciones y uno de datos, con lectura sincrónica de un ciclo), por lo que el resto del SoC no cambia. Se seleccionan desde `config.json` incluyendo la carpeta `rtl/mem_s1` o `rtl/mem_s2`.

4.3.1. Estrategia 1: ROM combinacional y RAM de datos reducida

Las 100 instrucciones se implementan en una ROM combinacional (`pochoco_rom.v`) generada por `assembler_v2.py`: una sentencia `case` indexada por la dirección de palabra (`addr[31:2]`), en la que las direcciones fuera del programa entregan un `nop`. A su salida se agrega un registro para mantener la lectura sincrónica de un ciclo. Los datos se almacenan en una RAM de Flip-Flops con escritura por bytes (`pochoco_ram.v`).

En la primera versión, con 16 palabras de datos (`p3_2_s1_v2`), la síntesis arrojó 205.699,0 μm^2 de área de celdas y 1.357 Flip-Flops, y la RAM, que el juego no usa, ocupaba cerca de un tercio del área. Por eso se redujo al mínimo válido del módulo, 2 palabras (`p3_2_s1_dw2`); con una sola palabra, `$clog2(1)` vale 0 y la dirección queda sin bits. El área de celdas bajó un 26,3 %, a 151.587,3 μm^2, el *die* a 406.970 μm^2 y los Flip-Flops a 909. La limitación de esta decisión es que no queda espacio para variables o *stack* de otros programas: se justifica solo porque el juego no usa RAM.

4.3.2. Estrategia 2: memoria unificada de Flip-Flops con reset y set

Esta estrategia implementa una memoria de 100 palabras de 32 bits (`p3_2_s2`), generada por la función `generar_mem_ff` de `assembler_v2.py`. El programa se carga como valor de reset de cada bit: los bits en 0 usan Flip-Flops con reset asíncrono (`dffrnq_1`) y los bits en 1, Flip-Flops con set asíncrono (`dffsnq_1`), de modo que cada reset del sistema vuelve a cargar el programa. La memoria sigue siendo escribible por el puerto de datos. Como está dimensionada según el uso real del juego, sus 100 palabras corresponden al programa y ninguna a datos. Los 3.200 bits del programa se implementaron con 2.176 celdas `dffrnq_1` y 1.024 celdas `dffsnq_1`.

El costo en recursos es alto: 635.477,5 μm^2 de área de celdas, 1.645.590 μm^2 de *die* y 4.045 Flip-Flops, con la memoria ocupando por sí sola el 79,2 % del área de celdas.

4.4. Comparación entre estrategias

La Tabla 3 compara las estrategias con $T = 100$ ns y $U = 40\,\%$.

Métrica	**Estrategia 1 (16 palabras)**	**Estrategia 1 (2 palabras)**	**Estrategia 2 (100 palabras)**	**Est. 2 / Est. 1 (2 palabras)**
Área de celdas (μm^2)	205.699,0	151.587,3	635.477,5	×4,19
Total de Flip-Flops	1.357	909	4.045	×4,45
Área del *die* (μm^2)	546.795	406.970	1.645.590	×4,04
Utilización final del core (%)	50,4	50,6	51,3	+0,7 pp
Setup WNS, corner lento (ns)	+44,77	+45,54	+6,90	−38,64
$f_{\text{máx}}$ estimada (MHz)	$\sim$ 18,1	$\sim$ 18,4	$\sim$ 10,7	×0,58

Tabla 3: Comparación entre las estrategias de memoria. La frecuencia estimada es $1/(T - \text{WNS})$ en el corner lento; el valor definitivo se obtiene en la Sección 6.

Fuente: Elaboración Propia.

La Estrategia 2 multiplica el área de celdas por 4,19 y los Flip-Flops por 4,45 respecto a la Estrategia 1 con RAM de 2 palabras. La causa es estructural: en la Estrategia 1 el programa reside en una ROM combinacional de 6.805,1 μm^2 (4,5 % del área de celdas), mientras que en la Estrategia 2 sus 3.200 bits se almacenan en Flip-Flops, con lo que el bloque de memoria pasa de 19.361,7 μm^2 (ROM más RAM de 2 palabras) a 503.409,8 μm^2. El tipo de celda es un factor secundario: un Flip-Flop con reset o set ocupa unos 74,6 μm^2, frente a 63,7 μm^2 de uno sin reset, un 17 % más.

La caída de la frecuencia estimada, de $\sim$ 18,4 a $\sim$ 10,7 MHz, se atribuye a los dos multiplexores de lectura de 100 a 1 y 32 bits (uno por puerto) que seleccionan la palabra leída. Esta red alarga la ruta de lectura y reduce el *slack* de setup de +45,54 a +6,90 ns.

Por lo anterior, se eligió la Estrategia 1 con RAM de 2 palabras como arquitectura definitiva: frente a la Estrategia 2, reduce el área de celdas en un 76 % y el *die* en un 75 %, con mejor desempeño temporal.

Las corridas de esta comparación se hicieron con el `espino_controller.v` original; las corridas posteriores incluyen la modificación de la Sección 6.4.

5. Síntesis y trazabilidad jerárquica

Los resultados de esta sección corresponden a la síntesis del diseño entregado (corrida `p3_4_T100`): Estrategia 1 con RAM de 2 palabras, `TenthCycles = 1000000` y el controlador modificado de la Sección 6.4, con $T = 100$ ns y $U = 40\,\%$. La síntesis es la misma en todas las corridas del barrido de período (Sección 6).

5.1. Método de trazabilidad

Por defecto, LibreLane aplana la jerarquía antes de sintetizar, de modo que el reporte solo mostraba `asic_top` y no era posible medir el área de cada módulo. Para evitarlo se fijó `SYNTH_HIERARCHY_MODE = deferred_flatten` en `config.json`, valor que se mantiene en todas las corridas siguientes. Con este modo, Yosys sintetiza cada módulo por separado, imprime su área (`Chip area for module`) y recién después aplana el diseño. Las áreas por módulo se obtuvieron de estas estadísticas previas al aplanamiento, en `yosys-synthesis.log` (líneas 9824 a 10291). Se usó la segunda tanda de estadísticas, porque la primera corresponde a una pasada intermedia incompleta. La suma de las áreas de los módulos (151.670,8 μm^2) coincide con el área final aplanada (151.424,9 μm^2) con una diferencia de 0,16 %.

Conservar la jerarquía tiene un costo: en la Estrategia 1 con 16 palabras, el modo `deferred_flatten` arrojó 205.165,6 μm^2, frente a 194.957,9 μm^2 con aplanado directo (+5,2 %), porque se pierden optimizaciones entre módulos.

Los bloques pedidos se identifican por sus módulos e instancias, cuyos nombres se conservaron:

- Banco de registros: módulo `espino_register_file`; instancia `u_pochoco.u_core.u_rf`.
- ALU: módulo `espino_alu`; instancia `u_pochoco.u_core.u_id.u_alu` (la ALU está instanciada dentro de la etapa de decodificación).
- Contador de ciclos: módulo `pochoco_cycle_counter`; instancia `u_pochoco.u_periph.u_cycle_counter`.

El contador de ciclos no existía como módulo en el Proyecto 2: era un registro de 32 bits (`reg [31:0] cycle_counter`) dentro de `pochoco_periph`, por lo que su área no podía medirse por separado. Se extrajo a `rtl/pochoco_cycle_counter.v`, con el mismo comportamiento, y se instanció en `pochoco_periph.v` como `u_cycle_counter`. Esta separación aumentó el área total en 533,4 μm^2 (+0,26 %), también por la pérdida de optimización entre módulos.

Después del aplanamiento, los nombres jerárquicos se conservan como prefijo de las instancias y redes del netlist (por ejemplo, `u_pochoco.u_core.u_rf._1393_`), lo que permite ubicar cada bloque en el layout (Sección 8).

5.2. Área por módulo

La Tabla 4 presenta el área de cada módulo antes del aplanamiento.

Módulo	**Área (μm^2)**	**% del total**
`espino_register_file`	74.085,8	48,9
`pochoco_ram` (RAM de datos y registros de salida)	12.556,5	8,3
`pochoco_periph`	11.098,9	7,3
`espino_alu`	10.152,8	6,7
`espino_id_stage`	10.021,1	6,6
`espino_if_stage`	8.510,8	5,6
`pochoco_rom`	6.805,1	4,5
`pochoco_spi_slave`	6.276,1	4,1
`pochoco_cycle_counter`	3.942,6	2,6
`espino_load_store_unit`	3.211,6	2,1
`pochoco_soc` (lógica propia)	2.717,7	1,8
`espino_decoder`	2.215,0	1,5
`espino_controller`	76,8	0,05
Suma	**151.670,8**	

Tabla 4: Área por módulo antes del aplanamiento (`p3_4_T100`). Cada valor excluye a sus submódulos; los porcentajes son respecto al área final aplanada (151.424,9 μm^2).
Fuente: Elaboración Propia.

El módulo de mayor área es `espino_register_file`, con 74.085,8 μm^2 (48,9 % del total): un banco de 15×32 Flip-Flops con dos puertos de lectura sincrónica de 32 bits. Su área no depende de la estrategia de memoria.

El contador de ciclos agregado en el Proyecto 2 ocupa 3.942,6 μm^2, el 2,6 % del área de celdas. Se compone de 32 Flip-Flops con reset asíncrono (`dffrnq_1`), que suman 2.388,4 μm^2 (61 %), y del incrementador, de unos 1.554 μm^2 (39 %). Su área absoluta es la misma en todas las corridas, pero su fracción depende del tamaño del resto del diseño: 1,9 % con la RAM de 16 palabras, 0,6 % con la Estrategia 2 y 2,6 % en el diseño entregado.

5.3. Celdas más usadas

La Tabla 5 presenta las cinco celdas más usadas en la síntesis y la parte del diseño que explica su cantidad. La distribución por módulo se obtuvo de las mismas estadísticas previas al aplanamiento. Entre las cinco suman cerca del 67 % del área de celdas.

Celda	**Cantidad**	**Área (μm^2)**	**Parte del diseño que la explica**
`mux2_2`	863	$\approx$ 28.420	551 en el banco de registros (64 %), 128 en `pochoco_ram` y 66 en `pochoco_soc`. Los Flip-Flops de la biblioteca no tienen entrada de habilitación, por lo que cada bit que solo se escribe a veces necesita un multiplexor que realimente su valor; en `pochoco_soc` seleccionan el dato leído (RAM, periféricos o SPI).
`nand2_1`	635	$\approx$ 6.970	Lógica combinacional general repartida: banco de registros (136), ALU (106), unidad de carga (94, alineación de bytes y medias palabras), decodificación (74), periféricos (63) y ROM (59).
`dffq_1`	608	$\approx$ 38.710	Flip-Flops sin reset: 480 del banco de registros (15×32) y 128 de `pochoco_ram` (64 de la RAM de datos de 2×32 y 64 de los registros de salida de ambos puertos).
`nor2_1`	388	$\approx$ 5.110	Lógica combinacional general: banco de registros (126), ROM (60), ALU (57), periféricos (55) y decodificación (45).
`dffrnq_1`	298	$\approx$ 22.240	Flip-Flops con reset asíncrono: periféricos (82: divisor de 22 bits, contador de décimas de 16, `rdata_o` de 32, LEDs de 4 y display de 8), *fetch* (65), registros de lectura del banco de registros (64), SPI (49), contador de ciclos (32), `pochoco_soc` (4: sincronizador de reset y selección de lectura), decodificación (1) y unidad de carga (1).

Tabla 5: Cinco celdas más usadas en la síntesis del diseño entregado (`p3_4_T100`). Áreas aproximadas, calculadas con el área unitaria de cada celda.

Fuente: Elaboración Propia.

El banco de registros explica buena parte de tres de las cinco celdas: aporta 480 de los 608 `dffq_1`, 551 de los 863 `mux2_2` y 64 de los 298 `dffrnq_1`. Esto es coherente con que sea el módulo de mayor área.

5.4. Comparación con lo esperado

Antes de revisar la síntesis de la Estrategia 1 (con RAM de 16 palabras) se registró una expectativa del resultado. La Tabla 6 la contrasta con lo obtenido.

Aspecto	**Expectativa antes de sintetizar**	**Resultado**
Flip-Flops frente a la memoria de 512 palabras (unos 16.384)	Reducción muy grande de Flip-Flops y de área.	Se cumplió: 1.355 Flip-Flops en todo el diseño; la corrida bajó de 1:35:33 a 0:18:39.
Flip-Flops sin reset	Banco de registros 480 + RAM de datos 512 + registros de salida de memoria 64 = 1.056.	Se cumplió: 1.056 `dffq_1` con síntesis jerárquica (1.054 con aplanado directo, por 2 bits que la optimización entre módulos elimina).
Módulos de mayor área	Banco de registros y RAM de datos, similares (480 y 512 FF); el banco algo mayor por sus multiplexores de lectura.	Se cumplió: 36,1 % y 32,3 % del área.
ROM	Menor que una memoria de Flip-Flops; su área depende de los patrones de bits.	Se cumplió: 3,4 % del área, unas 9,5 veces menos que la RAM de 16 palabras.
Contador de ciclos	Fracción pequeña del área.	Se cumplió: 1,9 %.
RAM de datos sin uso	No se eliminaría en la síntesis, porque Yosys no conoce el programa.	Se confirmó: cerca de un tercio del área y 512 de los 1.355 Flip-Flops, lo que motivó reducirla a 2 palabras.
Jerarquía	No se anticipó.	La síntesis aplanó el diseño por defecto; con `deferred_flatten` se midió cada módulo, a un costo de +5,2 % de área.

Tabla 6: Expectativa frente a resultado de la síntesis de la Estrategia 1.
Fuente: Elaboración Propia.

El único aspecto que no coincidió con lo esperado fue la jerarquía. Tampoco se anticipó que los multiplexores de realimentación fueran la celda más usada: como la biblioteca no tiene Flip-Flops con habilitación, cada registro que no se escribe en todos los ciclos agrega un `mux2_2` por bit. En el diseño entregado, con la RAM de 2 palabras, el banco de registros sigue siendo el módulo de mayor área, ahora con el 48,9 % del total, mientras que la RAM de datos baja de 32,3 % a 8,3 %.

6. Timing: barrido de período, ruta crítica y constantes del juego

Esta sección busca el menor período de reloj (`CLOCK_PERIOD`) con el que el diseño cierra *timing*: WNS ≥ 0 y TNS $= 0$ en setup y en hold. Luego identifica la ruta crítica, evalúa una modificación del RTL y calcula las constantes del juego para el período objetivo.

6.1. Barrido de `CLOCK_PERIOD`

Cada valor de `CLOCK_PERIOD` corresponde a una corrida completa e independiente (archivos `config_timing_*.json`), con el RTL final y el resto de la configuración sin cambios ($U = 40\,\%$). La Tabla 7 presenta el WNS y el TNS de setup y hold en el corner típico (`nom_tt_025C_5v00`) y en el más lento (`max_ss_125C_4v50`). Para decidir si una corrida cierra se usa además el WNS y TNS global, que es el peor caso de los nueve corners; en todas las corridas, el peor setup coincide con el corner lento.

T (ns)	Corner	Setup WNS (ns)	Setup TNS (ns)	Hold WNS (ns)	Hold TNS	¿Cierra?
100	típico	+72,31	0	+0,353	0	Sí
	lento	+46,89	0	+0,902	0	
60	típico	+32,31	0	+0,353	0	Sí
	lento	+6,89	0	+0,902	0	
55	típico	+27,31	0	+0,353	0	Sí
	lento	+1,89	0	+0,902	0	
54,5	típico	+26,81	0	+0,353	0	Sí
	lento	+1,39	0	+0,902	0	
53,2	típico	+25,51	0	+0,353	0	**Sí (mínimo)**
	lento	+0,093	0	+0,902	0	
50	típico	+22,31	0	+0,353	0	No
	lento	−3,107	−67,81	+0,902	0	

Tabla 7: Barrido de `CLOCK_PERIOD` con el RTL final ($U = 40\,\%$). Corridas `p3_4_T100`, `p3_4_T60`, `p3_4_T55`, `p3_4_T54_5`, `p3_4_T53_2` y `p3_4_T50`. El peor hold global es +0,118 ns en todas las corridas, en un corner distinto del típico y del lento.

Fuente: Elaboración Propia.

El resultado muestra que, en este rango de períodos, el flujo produjo el mismo circuito en todas las corridas: el área de celdas (191.204 μm^2), el área del *die* (406.549 μm^2) y el hold son idénticos, y el *slack* de setup varía exactamente lo mismo que el período. En el corner lento, $T - \text{WNS} = 53{,}107$ ns en las seis corridas, que es el retardo de la ruta crítica. Aunque cambiar `CLOCK_PERIOD` puede provocar una nueva optimización, con cambios en el área, el *buffering* y el *placement*, en estas corridas no ocurrió: la síntesis no se guía por el *timing*, y la reparación de *timing* del *place and route* no modificó el circuito, ni siquiera con 50 ns, donde el setup no cierra.

6.2. Frecuencia máxima

La frecuencia máxima se define con el menor período que cierra setup y hold en todos los corners exigidos. De la Tabla 7, ese período es $T_{\text{mín}} = 53{,}2$ ns, con un WNS de setup de +0,093 ns en el corner lento:

$$f_{\text{máx}} = \frac{1}{T_{\text{mín}}} = \frac{1}{53{,}2 \text{ ns}} \approx 18{,}80 \text{ MHz}. \tag{1}$$

Con $T = 50$ ns el diseño no cierra setup (WNS = −3,107 ns, TNS = −67,81 ns). Como el retardo de la ruta crítica es de 53,107 ns en el corner lento, el límite teórico es de unos 18,83 MHz; el período de 53,2 ns queda a menos de 0,1 ns de ese límite. El hold no restringe la frecuencia: su peor *slack* es positivo (+0,118 ns) en todas las corridas, porque no depende del período.

6.3. Ruta crítica

La ruta crítica se obtuvo del reporte de STA posterior al *place and route* de la corrida `p3_4_T53_2`, en el corner lento (`55-openroad-stapostpnr/max_ss_125C_4v50/max.rpt`). Como el circuito es el mismo en todo el barrido, es la misma ruta en todas las corridas. La Tabla 8 resume sus datos y la Tabla 9 la divide por módulo.

Dato	**Valor**
Corrida y corner	`p3_4_T53_2`, `max_ss_125C_4v50`
Celda de inicio	`u_pochoco.u_ram._346_` (`dffq_1`): bit 0 del registro de salida de la memoria de instrucciones (`instr_rdata[0]`)
Celda de fin	`u_pochoco.u_periph._622_` (`dffrnq_1`): bit del registro de lectura de periféricos (`rdata_o`) de `pochoco_periph`
Módulos recorridos	`pochoco_ram`, `espino_decoder`, `espino_id_stage`, `espino_alu`, `espino_load_store_unit`, `pochoco_soc`, `pochoco_periph`
Función microarquitectónica	Acceso a memoria (lectura de un periférico mapeado en memoria), que atraviesa además la decodificación y la ejecución
Llegada / requerido / *slack*	55,066 ns / 55,160 ns / +0,093 ns ($T = 53{,}2$ ns)

Tabla 8: Ruta crítica del diseño.
Fuente: Elaboración Propia.

Tramo	Módulo	Retardo (ns)
Reloj a salida del registro de instrucción	`pochoco_ram`	1,72
Decodificación (`alu_op_a_sel`)	`espino_decoder`	6,80
Selección del operando A (`operand_a`)	`espino_id_stage`	9,95
Suma de la dirección hasta el bit 31	`espino_alu`	18,64
Dirección a `data_addr[31]` y `per_req`	`espino_load_store_unit`, `pochoco_soc`	2,06
Habilitación y dato del registro de lectura	`pochoco_periph`	13,27
Total (desde el pin de reloj del registro de inicio)		**52,44**

Tabla 9: Retardo de la ruta crítica por tramo (corner lento).
Fuente: Elaboración Propia.

La ruta comienza en el registro que entrega la instrucción leída de la ROM. La instrucción se decodifica, se selecciona el operando A y la ALU calcula la dirección del acceso a memoria (`x2` más un desplazamiento). El bit 31 de esa dirección, que es el último que produce la cadena de acarreo de la suma, indica en `pochoco_soc` que el acceso va a los periféricos (`is_mmio` y luego `per_req`). En `pochoco_periph`, esa señal habilita la actualización del registro de lectura `rdata_o` (`if (access & ~we_i)`), cuya habilitación se distribuye a los 32 bits mediante un árbol de tres niveles de *buffers* (9,5 ns). Todo ocurre en un solo ciclo, por lo que la ruta encadena las etapas de decodificación, ejecución y acceso a memoria.

Listado 2: Extracto de la ruta crítica (`max.rpt`, corner lento). Fuente: Elaboración Propia.

```
Startpoint: u_pochoco.u_ram._346_
Endpoint: u_pochoco.u_periph._622_
 ... 4.349 ^ u_pochoco.u_ram._346_/Q (dffq_1) instr_rdata[0]
 ... 11.149 ^ u_pochoco.u_core.u_id.u_decoder._108_/ZN alu_op_a_sel[0]
 ... 21.099 v u_pochoco.u_core.u_id._805_/ZN operand_a[1]
 ... 39.736 ^ u_pochoco.u_core.u_id.u_alu._1133_/ZN lsu_addr[31]
 ... 41.794 ^ u_pochoco._057_/ZN per_req
 ... 55.066 v u_pochoco.u_periph._622_/D (dffrnq_1)
    55.160 data required time -> slack = +0.093 ns
```

Llama la atención que los seis *buffers* de *fanout* de la ruta, insertados por el flujo durante la reparación del diseño, son celdas de retardo (`dlyb_1`). En conjunto aportan 19,7 ns, el 37,6 % de la ruta. Esto lo decide el flujo y no el RTL, pero indica que la ruta podría acortarse también desde la configuración de LibreLane, excluyendo esas celdas del *buffering*.

6.4. Modificación del RTL

6.4.1. Primer intento: señal `pc_set_o` del controlador

El primer cambio evaluado fue en `espino_controller.v`: la señal `pc_set_o`, que indica al *fetch* que debe cargar la dirección de un salto, pasó a reutilizar `stall_o` en vez de volver a combinar `rf_ready_i` y `lsu_busy_i` (Listado 3). Este cambio forma parte del RTL entregado.

Listado 3: Cambio en `espino_controller.v`. Fuente: Elaboración Propia.

```
// Original (Proyecto 2)
assign pc_set_o = instr_valid_i & rf_ready_i
                & ~lsu_busy_i
                & (branch_taken_i | jump_i);

// Modificado (stall_o = (instr_valid_i & ~rf_ready_i) | lsu_busy_i)
assign pc_set_o = instr_valid_i & ~stall_o
                & (branch_taken_i | jump_i);
```

Ambas expresiones son lógicamente equivalentes. Con $v =$ `instr_valid_i`, $r =$ `rf_ready_i` y $l =$ `lsu_busy_i`:

$$v \cdot \overline{(v \cdot \overline{r}) + l} = v \cdot (\overline{v} + r) \cdot \overline{l} = v \cdot r \cdot \overline{l}.$$

Como Yosys optimiza la lógica booleana, dos expresiones equivalentes producen esencialmente el mismo circuito. Además, `pc_set_o` no forma parte de la ruta crítica del diseño (Tabla 8), por lo que este cambio no podía acortarla. La diferencia de *slack* observada respecto a `p3_2_s1_dw2` (+45,54 frente a +46,89 ns con $T = 100$ ns) se atribuye a variaciones de síntesis y *placement* entre corridas, no al cambio en sí.

6.4.2. Cambio propuesto: lectura de periféricos sin habilitación

La ruta crítica termina en el registro de lectura `rdata_o` de `pochoco_periph`, que solo se actualiza cuando hay una lectura de periférico (`access & ~we_i`). Esa habilitación depende de `per_req`, que a su vez depende del bit 31 de la dirección: el último bit que produce la suma de la ALU. Luego se distribuye a los 32 bits con un árbol de *buffers*. El cambio propuesto es actualizar `rdata_o` en todos los ciclos según la dirección, eliminando esa dependencia (Listado 4).

Listado 4: Cambio en `pochoco_periph.v (línea 109)` `Fuente: Elaboración Propia..`

```
// Original
else if (access & ~we_i) begin
// Modificado
else begin // 3.4: rdata_o se actualiza en todos los ciclos
```

Con el cambio, `rdata_o` solo depende de `addr_i[7:2]`, que proviene de los bits bajos de la suma y se resuelve mucho antes que el bit 31. El comportamiento no cambia: la unidad de carga pide el dato en un ciclo y lo usa en el siguiente, y `pochoco_soc` solo selecciona `rdata_o` cuando la carga anterior fue a un periférico (`rd_is_per`). En ese ciclo, `rdata_o` tiene el valor de la dirección de esa carga, igual que antes. Leer un periférico tampoco tiene efectos secundarios, por lo que actualizarlo en otros ciclos no altera nada.

El cambio se implementó en una copia del diseño y se evaluó con las mismas configuraciones de $T = 100$ y 53,2 ns (corridas `p3_4_opt_T100` y `p3_4_opt_T53_2`). La Tabla 10 compara los resultados.

Métrica	**Original**	**Con el cambio**	**Variación**
Setup WNS lento, $T = 53{,}2$ ns (ns)	+0,093	+1,661	+1,568
Setup WNS lento, $T = 100$ ns (ns)	+46,89	+48,46	+1,568
Retardo de la ruta crítica, $T -$ WNS (ns)	53,107	51,539	−1,568 (−2,95 %)
Frecuencia límite, $1/(T - \text{WNS})$ (MHz)	18,83	19,40	+3,0 %
Área de celdas, síntesis (μm^2)	151.424,9	150.718,0	−706,9 (−0,47 %)
Área de celdas, final (μm^2)	191.204	190.045	−1.159 (−0,61 %)
Área del *die* (μm^2)	406.549	404.715	−1.834 (−0,45 %)
Total de Flip-Flops	909	909	0

Tabla 10: Efecto de la lectura de periféricos sin habilitación. Corridas `p3_4_T100` y `p3_4_T53_2` (original) frente a `p3_4_opt_T100` y `p3_4_opt_T53_2` (con el cambio).
Fuente: Elaboración Propia.

El cambio mejora el *slack* en 1,568 ns en ambos períodos y reduce levemente el área, porque los 32 bits de `rdata_o` ya no necesitan la lógica que conserva su valor cuando no hay lectura. La mejora es igual a 100 y 53,2 ns, lo que confirma que se debe al cambio del RTL y no al período.

La mejora es moderada porque, al sacar `rdata_o` de la ruta, la ruta crítica pasa a ser otra que ya era casi igual de larga. La nueva ruta (Tabla 11) parte del mismo registro de instrucción y comparte la decodificación y la selección del operando, pero la ALU calcula la comparación de un *branch* (`alu_cmp_result`), que pasa por `branch_taken` y por el controlador (`pc_set`) hasta un registro de la etapa de *fetch* (`u_pochoco.u_core.u_if._372_`). Su función es resolver un salto y cargar la nueva dirección en el mismo ciclo, y también termina en un árbol de tres *buffers* de *fanout* (8,9 ns). Para acortarla habría que resolver los saltos en una etapa posterior, lo que cambiaría la microarquitectura del *pipeline*.

Tramo	**Módulo**	**Retardo (ns)**
Reloj a salida del registro de instrucción	`pochoco_ram`	1,74
Decodificación	`espino_decoder`	7,80
Selección del operando A (`operand_a[23]`)	`espino_id_stage`	11,17
Comparación del *branch* (`alu_cmp_result`)	`espino_alu`	13,78
`branch_taken` a `pc_set`	`espino_id_stage`, `espino_controller`	3,29
Siguiente PC y su registro	`espino_if_stage`	12,74
Total (desde el pin de reloj del registro de inicio)		**50,52**

Tabla 11: Nueva ruta crítica con el cambio (`p3_4_opt_T53_2`, corner lento; llegada 53,269 ns, requerido 54,930 ns, *slack* +1,661 ns).
Fuente: Elaboración Propia.

El cambio no se incorporó al macro entregado: hacerlo habría obligado a repetir la corrida final, el signoff, el barrido de utilización y la simulación post-síntesis, que se hicieron con el RTL anterior. Por eso, la frecuencia máxima del diseño entregado sigue siendo la de la Sección 6 (18,80 MHz). Con el cambio, la frecuencia límite estimada sería de unos 19,40 MHz.

6.5. Constantes del juego para el período T

El programa `game.s` no contiene la frecuencia del reloj. Mide el tiempo con el contador de décimas de segundo del periférico (`tenth_counter`), que se incrementa cada `TENTH_CYCLES` ciclos, y espera tres segundos comparando con un umbral de 30 décimas (registro `x12`). La relación entre ciclos y tiempo está, por lo tanto, en el parámetro de hardware:

$$\text{TENTH_CYCLES} = \frac{0{,}1 \text{ s}}{T} = \frac{0{,}1 \text{ s}}{100 \text{ ns}} = 10^6 \text{ ciclos.} \tag{2}$$

Con el reloj de 25 MHz de la FPGA ($T = 40$ ns) el valor era 2.500.000. La espera de tres segundos son 30 décimas, es decir, 3×10^7 ciclos a 10 MHz.

Constante	**Proyecto 2 (25 MHz)**	**ASIC ($T = 100$ ns)**	**Ubicación**
`TENTH_CYCLES`	2.500.000	1.000.000	Hardware (`asic_top.v`)
Umbral de espera de 3 s (`x12`)	30	30	`game.s`
Divisor y redondeo del promedio	10 y 5	10 y 5	`game.s`
Vuelta del contador de ciclos (32 bits)	171,8 s	429,5 s	Informativo

Tabla 12: Constantes que dependen del período del reloj.
Fuente: Elaboración Propia.

Como el programa expresa todos los tiempos en décimas de segundo, el umbral de 30 y los cálculos del promedio no dependen del período del reloj: la única constante que cambia es `TENTH_CYCLES`, el supuesto de la FPGA identificado en la Sección 3. Por eso, la respuesta a qué constantes de `game.s` deben cambiar para que la espera siga siendo de tres segundos y el tiempo se muestre en décimas es que ninguna instrucción del programa cambia: el ajuste se hace en el parámetro de hardware, que pasa de 2.500.000 a 1.000.000 de ciclos.

La alternativa de mantener `TENTH_CYCLES` = 2.500.000 y modificar el programa no cumpliría el requisito. A 10 MHz, el contador avanzaría cada $2.500.000 \times 100$ ns = 0,25 s, por lo que la espera de tres segundos requeriría cambiar el umbral de 30 a 12, pero el tiempo de reacción solo se podría medir en pasos de 0,25 s. Ningún cálculo en `game.s` puede recuperar una resolución de décimas que el contador no mide. Como nota adicional, $10^6 < 2^{20}$, por lo que el divisor de 22 bits podría reducirse a 20 sin afectar el funcionamiento.

En `game.s` solo se actualizaron los comentarios que describen la frecuencia del reloj (líneas 14 y 15: 10 MHz y 1.000.000 de ciclos por décima). El programa se regeneró con `assembler_v2.py` a partir de ese archivo, y tanto `game.hex` como la ROM generada son idénticos a los obtenidos con el programa del Proyecto 2, lo que confirma que el código ejecutable no cambió.

7. Floorplan, red de alimentación y placement

7.1. Barrido de utilización del core

Se ejecutó el flujo con `FP_CORE_UTIL` = 40, 50, 60, 70, 72, 73, 74, 75 y 80 % (`config_util_*.json`), con $T = 100$ ns y el resto de la configuración sin cambios. LibreLane calcula el área del core a partir del área sintetizada y la utilización objetivo. Una corrida se considera cerrada con los criterios del proyecto: flujo completo, DRC y LVS limpios, sin violaciones de antena, y setup y hold con WNS ≥ 0 y TNS $= 0$. La Tabla 13 resume los resultados (archivos en `evidencia_3_5/metricas`).

Config. (%)	**Util. final (%)**	**Die (μm^2)**	**Ruteo (mm)**	**Slew / cap**	**WNS setup / hold (ns)**	**Estado**
40	50,9	406.549	329,2	1.718 / 1	+46,9 / +0,12	Completa
50	62,4	327.928	312,8	1.582 / 3	+45,5 / +0,13	Completa
60	74,1	275.303	310,6	1.589 / 2	+51,8 / +0,14	Completa
70	86,2	237.581	401,6	2.226 / 11	+44,2 / +0,18	Completa
72	88,9	231.281	471,3	2.698 / 35	+45,7 / +0,17	Completa
73	89,4	228.256	469,5	2.735 / 22	+35,3 / +0,18	**Completa (mínimo)**
74	—	—	—	—	—	Falla en CTS
75	—	—	—	—	—	Falla en CTS
80	—	—	—	—	—	Falla en *placement*

Tabla 13: Barrido de `FP_CORE_UTIL` ($T = 100$ ns). "Slew / cap" es el número de violaciones de *max slew* y *max cap*. El TNS de setup y hold es 0 en todas las corridas completas.
Fuente: Elaboración Propia.

Las seis corridas completas terminan con 0 errores DRC (Magic), 0 errores LVS y 0 redes con violación de antena. El área mínima con la que el diseño completa el flujo es la de 73 %: un *die* de 228.256 μm^2 (0,228 mm^2), un 43,9 % menor que el de 40 %. Con 74 % y 75 % el flujo falló en la síntesis del árbol de reloj (CTS), y con 80 %, en el *placement*.

Las tres corridas fallidas terminan con el mismo error de OpenROAD, `[ERROR DPL-0036] Detailed placement failed`: el legalizador no encuentra espacio libre para ubicar las celdas que el flujo agrega. Con 74 % y 75 %, el error ocurre en la síntesis del árbol de reloj, al ubicar los *buffers* de reloj insertados (2 y 13 instancias sin ubicar, respectivamente). Con 80 %, ocurre antes, en la reparación del diseño posterior al *placement* global, al insertar *buffers* de *fanout* y de *slew*. La Tabla 14 indica el registro de cada falla.

Config. (%)	Etapa	Evidencia (archivo, línea)
74	CTS (`OpenROAD.CTS`)	`runs/p3_5_U74/35-openroad-cts/openroad-cts.log`, línea 118
75	CTS (`OpenROAD.CTS`)	`runs/p3_5_U75/35-openroad-cts/openroad-cts.log`, línea 130
80	Reparación del diseño (`OpenROAD.RepairDesignPostGPL`)	`runs/p3_5_U80/32-openroad-repairdesignpostgpl/openroad-repairdesignpostgpl.log`, línea 87

Tabla 14: Etapa y registro de las corridas que no completaron el flujo. En todas el error es `DPL-0036`.

Fuente: Elaboración Propia.

Como la utilización final supera a la configurada en unos 16 puntos (Tabla 13), con 74 % el core quedaría sobre el 90 % de ocupación antes del árbol de reloj, sin espacio para sus *buffers*. Por eso el área mínima viable es la del 73 %.

El *die* de la corrida de 40 % (406.549 μm^2) difiere en 0,1 % del de `p3_2_s1_dw2` (406.970 μm^2) porque el barrido usa el RTL entregado, con el controlador modificado de la Sección 6.4. Sus métricas son idénticas a las de `p3_4_T100` (Sección 6), que usa ese mismo RTL.

La utilización final es entre 11 y 17 puntos porcentuales mayor que la configurada, porque después de la síntesis el flujo agrega *buffers*, el árbol de reloj, *tap cells* y celdas de reparación de *timing*. Por eso, 73 % configurado equivale a 89,4 % real.

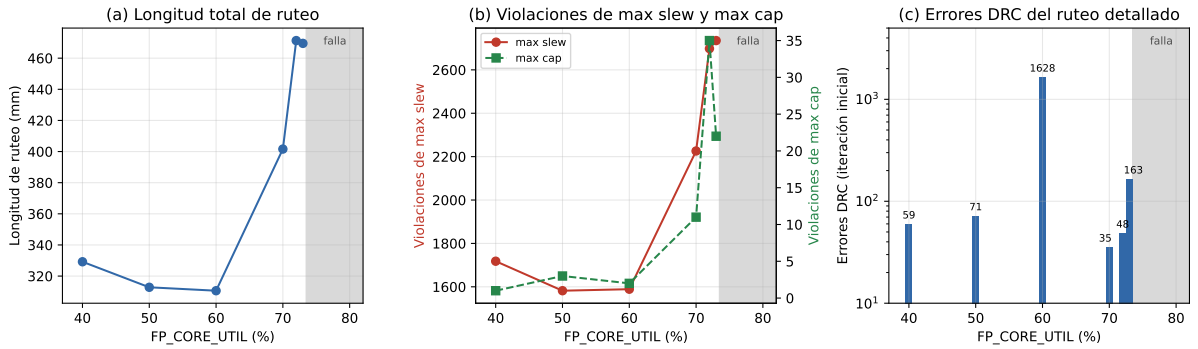


Figura 1: Métricas del flujo frente a `FP_CORE_UTIL`: (a) longitud total de ruteo, (b) violaciones de *max slew* y *max cap*, (c) errores DRC en la primera iteración del ruteo detallado. La zona sombreada marca las utilizaciones en que el flujo falló.

Fuente: Elaboración Propia.

Como métrica de congestión, la longitud total de ruteo (Figura 1a) se mantiene en torno a 311–313 mm hasta 60 %, sube a 401,6 mm en 70 % (+29 %) y llega a unos 470 mm en 72 % y 73 % (+51 %): con menos espacio libre, las rutas se alargan para esquivar celdas. En paralelo, las violaciones de *max slew* pasan de 1.589 (60 %) a 2.735 (73 %), las de *max cap* de 2 a 22–35, y el margen de setup baja de +51,8 a +35,3 ns. Los errores DRC de la primera iteración del ruteo detallado no siguen una tendencia (59, 71, 1.628, 35, 48 y 163) y todos convergen a 0. El pico de 60 % muestra que la dificultad de ruteo depende del *placement* concreto y no crece de forma monótona con la densidad. El límite de 73 % es un límite práctico del flujo con pasos de 1 %, no un límite físico.



Figura 2: Layout de la corrida con `FP_CORE_UTIL` = 73 % (todas las capas).
Fuente: Elaboración Propia.

7.2. Red de alimentación

El `config.json` no define variables de la red de alimentación, por lo que se usó la configuración por defecto de LibreLane para `gf180mcuD`. La red se genera en la etapa `21-openroad-generatepdn`, y sus parámetros, obtenidos de `resolved.json` de la corrida `p3_4_T100`, se resumen en la Tabla 15.

Elemento	**Capa**	**Ancho (μm)**	**Separación (μm)**	**Paso (μm)**	**Offset (μm)**
Rieles de las filas de celdas	Metal1	0,6	—	Fila de celdas	0
Franjas verticales	Metal4	1,6	1,7	153,60	16,32
Franjas horizontales	Metal5	1,6	1,7	153,18	16,65

Tabla 15: Red de alimentación por defecto (`PDN_*` en `resolved.json`). La separación es entre la franja de VDD y la de VSS.
Fuente: Elaboración Propia.

Los rieles de Metal1 recorren cada fila de celdas y alimentan directamente a las celdas estándar, que se ubican tocándolos. Las franjas de Metal4 y Metal5 forman una malla que distribuye VDD y VSS por todo el macro y se conecta a los rieles mediante vías. No se generó anillo de alimentación (`PDN_CORE_RING = false`), lo que es habitual en un macro: al integrarlo en un chip, su malla se conectaría a la red de alimentación del nivel superior a través de los pines VDD y VSS que expone (`PDN_ENABLE_PINS = true`).

El efecto sobre el área utilizable es pequeño. Los rieles ya forman parte de la altura de cada fila de celdas, y las franjas van en capas altas, por encima de las celdas, por lo que no quitan espacio de *placement*. Su costo es en recursos de ruteo: cada par VDD/VSS ocupa 3,2 μm de cada 153,6 μm, cerca del 2 % de las pistas de Metal4 y Metal5. El flujo advierte que `VSRC_LOC_FILES` no está definido, por lo que el análisis de caída IR no es representativo; la propia advertencia indica que puede ignorarse cuando no se integra un chip completo, como en este caso.

7.3. Terminales del macro

Los terminales del macro son formas de metal en el borde del área de diseño donde terminan las redes de entrada y salida; no incluyen circuitería. Un chip completo necesitaría, en cambio, *pads* de I/O: celdas de mayor tamaño con protección ESD y *buffers* capaces de manejar cargas externas (LEDs, displays y líneas de la placa), conexión al encapsulado mediante *bond pads* y *pads* propios de alimentación. En este diseño esos elementos quedan fuera del macro.

Los terminales los ubicó automáticamente OpenROAD (etapa `25-openroad-ioplacement`), sin un archivo de orden de pines. La herramienta coloca cada terminal en el borde más cercano a la lógica a la que se conecta, para acortar el cableado. El *die* mide 628,7 × 646,6 μm, y la Tabla 16 indica dónde quedó cada grupo, según el DEF final de `p3_4_T100`.

Grupo	Dir.	Ancho	Borde y posición	Relación con su función
`btn_i`	in	4	Este, $y \approx$ 102–128 μm	Entradas leídas por `pochoco_periph`; quedan junto al resto de las señales de la interfaz con el usuario.
`led_o`	out	4	Este, $y \approx$ 260–275 μm	Salidas de `pochoco_periph`; agrupadas con los displays.
`seg1_o`, `seg2_o`	out	14	Este, $y \approx$ 263–317 μm	Salidas de `pochoco_periph`; intercaladas con los LEDs, que provienen del mismo módulo.
SPI (`sclk`, `cs_n`, `miso`)	in/out	3	Este, $y \approx$ 172–183 μm	Interfaz de `pochoco_spi_slave`, agrupada en un solo tramo del borde.
`spi_mosi_i`	in	1	Sur, $x \approx 528$ μm	Separada del resto del bus SPI.
`clk_i`	in	1	Oeste, $y \approx 594$ μm	Entra a la raíz del árbol de reloj, que la distribuye a todo el macro; su posición tiene poco efecto en el *skew*.
`rst_ni`	in	1	Sur, $x \approx 321$ μm	Va al sincronizador de reset; entrada lenta y asíncrona.

Tabla 16: Ubicación de los terminales del macro (DEF final de `p3_4_T100`).
Fuente: Elaboración Propia.

La ubicación automática sigue la función de los terminales solo en parte. Las señales de la interfaz con el usuario (botones, LEDs y displays) quedan juntas en la mitad inferior del borde este, cerca de `pochoco_periph`, y el reloj queda aislado en el borde oeste, lo que no es un problema porque el árbol de reloj lo distribuye desde su raíz. En cambio, el bus SPI quedó dividido entre dos bordes y el reset en un tercero. Para un chip real convendría fijar el orden con un archivo de configuración de pines (`FP_PIN_ORDER_CFG`), agrupando cada interfaz en un solo borde para facilitar el ruteo hacia los *pads* y la placa.

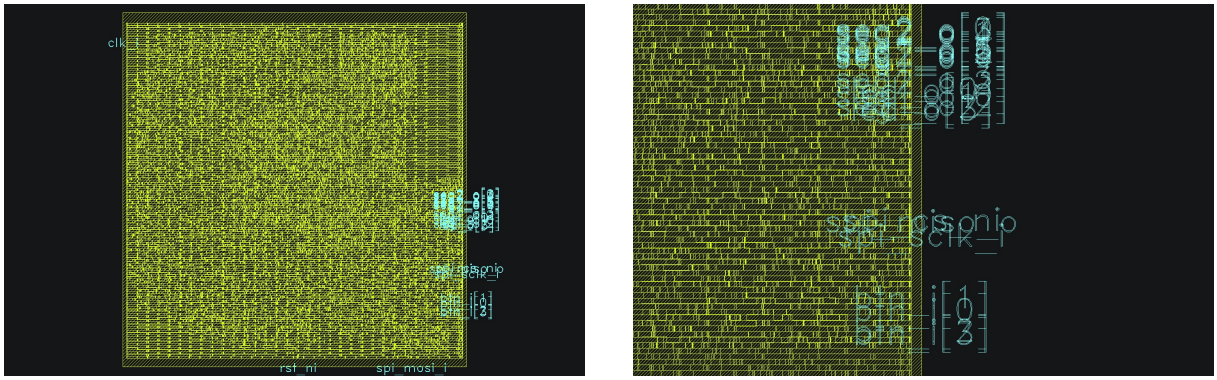


Figura 3: Terminales del macro en el layout final (`p3_4_T100`). Izquierda: macro completo, con `clk_i` en el borde oeste y `rst_ni` y `spi_mosi_i` en el borde sur. Derecha: detalle del borde este, con los botones, el bus SPI, los LEDs y los displays.

Fuente: Elaboración Propia.

7.4. Costo estimado

Con el dato visto en clases (US$23.000 por mm^2) y suponiendo, solo para este ejercicio, que el costo escala linealmente con el área del *die*, la Tabla 17 estima el costo de las configuraciones evaluadas.

Configuración	**Die (μm^2)**	**Die (mm^2)**	**Costo (US$)**
Estrategia 2 (100 palabras), $U = 40\,\%$	1.645.590	1,646	37.849
Estrategia 1 (16 palabras), $U = 40\,\%$	546.795	0,547	12.576
Estrategia 1 (2 palabras), $U = 40\,\%$	406.549	0,407	9.351
Estrategia 1 (2 palabras), $U = 73\,\%$	228.256	0,228	5.250

Tabla 17: Costo equivalente con costo lineal de US$23.000 por mm^2.
Fuente: Elaboración Propia.

La aproximación tiene limitaciones importantes. El costo real no es lineal: hay costos fijos (máscaras y áreas mínimas en corridas compartidas), el rendimiento de fabricación cae con el área, y el encapsulado y las pruebas se suman al silicio. Además, el macro no incluye *pads* de I/O, anillo de sellado ni la red de alimentación de un chip completo, por lo que su área subestima la de un chip real. Por último, operar al 89 % de utilización deja poco margen para cambios posteriores.

De las decisiones tomadas, la que más pesó fue la estrategia de memoria: elegir la Estrategia 1 en vez de la 2 reduce el *die* en un factor 4,05 (de US$37.849 a US$9.351). Le sigue subir la utilización de 40 % a 73 %, que lo reduce en un factor 1,78 (de US$9.351 a US$5.250), y luego reducir la RAM de 16 a 2 palabras, con un factor 1,34 (de US$12.576 a US$9.351). En conjunto, la configuración de menor área cuesta unas 7 veces menos que la Estrategia 2 con $U = 40\,\%$. La corrida final se entrega con $U = 40\,\%$, como fija el enunciado.

8. Signoff y layout

8.1. Resultados de DRC, LVS y antena

La Tabla 18 reúne los resultados de la corrida final (`andres_final_100`), con $T = 100$ ns y $U = 40\,\%$. Los archivos están en `evidencia_3_6`.

Verificación	**Herramienta**	**Resultado**	**Evidencia**
DRC	Magic	0 errores	`drc.rpt`
DRC	KLayout	Omitido (ver nota)	—
LVS	Netgen	*Circuits match uniquely*	`lvs.rpt`

Antena	OpenROAD	10 violaciones detectadas; 0 después de la reparación (12 *jumpers*)	`antena_antes.rpt`
			`antena_despues.rpt`
			`reparacion_antena.log`

Tabla 18: Resultados finales de signoff.
Fuente: Elaboración Propia.

El DRC de KLayout no se ejecutó porque la variable `KLAYOUT_DRC_RUNSET` no está definida para `gf180mcuD` en LibreLane 3.0.17; la verificación de reglas de diseño se hizo con Magic.

La corrida `andres_final_100` implementa el RTL entregado. Para verificarlo, se repitió la corrida con el RTL del repositorio (`p3_4_T100`): ambas tienen 909 celdas secuenciales y el mismo número de celdas en cada uno de los bloques identificados en el layout (1.965, 589 y 114; ver más adelante en esta sección).

8.2. Violación de antena detectada y corregida

Después del ruteo global, OpenROAD detectó 10 violaciones de antena (`GRT-0012`). Las redes afectadas (`antena_antes.rpt`) pertenecen principalmente al banco de registros: por ejemplo, `u_pochoco.u_core.u_rf._1393_`, con una razón parcial de 865,75 frente a un límite de 400 (2,16 veces) en Metal2, y las direcciones de lectura `rf_raddr_a[3]` y `rf_raddr_b[1]`. Son redes de alto *fanout* que recorren un bloque grande hasta los *buffers* insertados por el flujo, por lo que acumulan mucho metal conectado a compuertas.

El efecto de antena ocurre cuando un tramo largo de metal conectado solo a compuertas de transistores acumula carga durante el grabado con plasma, lo que puede dañar el óxido de compuerta. La etapa de reparación del flujo ejecutó `repair_antennas` (3 iteraciones, margen de 10 %), que insertó 12 *jumpers* en las 10 redes: el tramo largo se interrumpe y continúa en una capa superior, de modo que en cada paso de fabricación el metal conectado a la compuerta es menor. No se insertaron diodos. Después de la reparación, el flujo reportó 0 violaciones y `antena_despues.rpt` quedó vacío.

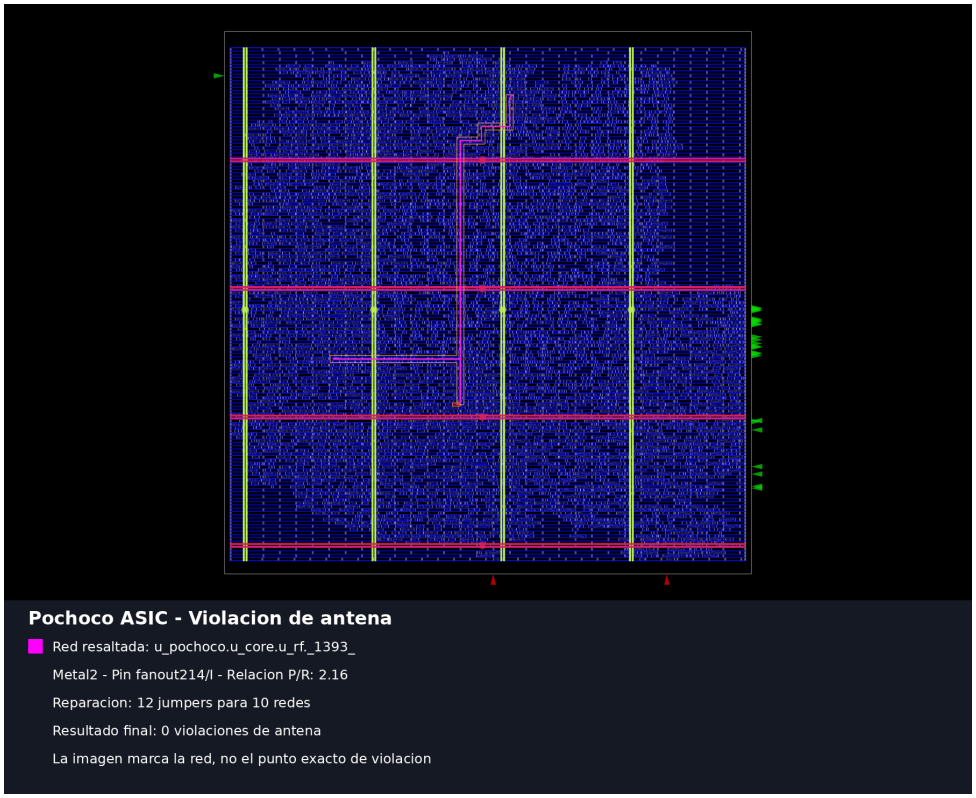


Figura 4: Red con violación de antena en el layout. La imagen marca la red y los datos de la reparación, no el pin exacto.

Fuente: Elaboración Propia.

Listado 5: Extracto de `reparacion_antena.log`. Fuente: Elaboración Propia.

```
+ repair_antennas gf180mcu_fd_sc_mcu7t5v0__antenna -iterations 3 -ratio_margin 10
[INFO GRT-0012] Found 10 antenna violations.
[INFO GRT-0302] Inserted 12 jumpers for 10 nets.
[INFO GRT-0012] Found 0 antenna violations.
```

8.3. Violaciones de *max slew* y *max cap*

Las corridas reportan violaciones de *max slew* y de *max cap*. No forman parte de los criterios de cierre del enunciado, pero se analizaron con el reporte de verificaciones posterior al *place and route* de `p3_4_T100` (`checks.rpt` de cada corner). La Tabla 19 muestra su distribución por corner.

Corner (proceso, temperatura, voltaje)	Parásitos mín.	Parásitos nom.	Parásitos máx.
Típico (tt, 25 °C, 5,0 V)	0 / 1	0 / 1	22 / 1
Lento (ss, 125 °C, 4,5 V)	1.041 / 1	1.373 / 1	1.718 / 1
Rápido (ff, −40 °C, 5,5 V)	0 / 1	0 / 1	0 / 1

Tabla 19: Violaciones de *max slew* / *max cap* por corner (`p3_4_T100`).
Fuente: Elaboración Propia.

Las violaciones de *max slew* se concentran en el corner lento: en el típico y en el rápido no hay, salvo 22 en el típico con parásitos máximos. La peor transición es de 5,46 ns frente a un límite de 3,0 ns. Las redes afectadas pertenecen principalmente al banco de registros (`u_pochoco.u_core.u_rf`) y a los *buffers* de *fanout* que el flujo insertó para manejarlas (Listado 6): son las señales de alto *fanout* que controlan la escritura y lectura de sus 480 bits, las mismas que causaron las violaciones de antena. El reporte indica además 58 violaciones de *max fanout*. La única violación de *max cap* está en `clkbuf_0_clk_i`, el *buffer* raíz del árbol de reloj, que maneja 0,361 pF frente a un límite de 0,200 pF; como la capacitancia no depende del corner, aparece en los nueve.

Listado 6: Peores violaciones de *max slew* y *max cap* (`checks.rpt`, corner lento).

```
Pin Limit Slew Slack
fanout73/I 3.000 5.459 -2.459 (VIOLATED)
u_pochoco.u_core.u_id._710_/ZN 3.000 5.459 -2.459 (VIOLATED)
u_pochoco.u_core.u_rf._2237_/C 3.000 5.250 -2.250 (VIOLATED)
fanout438/I 3.000 5.250 -2.250 (VIOLATED)
...
Pin Limit Cap Slack
clkbuf_0_clk_i/Z 0.200 0.361 -0.161 (VIOLATED)
```

Fuente: Elaboración Propia.

El grupo decidió justificar estas violaciones en lugar de corregirlas, por tres razones. Primero, no están entre los criterios de cierre del enunciado. Segundo, el análisis de *timing* ya incluye estas transiciones lentas en el cálculo de los retardos, y aun así el diseño cierra setup y hold en todos los corners, con un margen de setup de +46,89 ns a $T = 100$ ns. Tercero, aparecen casi exclusivamente en el corner lento, lo que es coherente con que la reparación de diseño del flujo no las haya detectado como violaciones en las condiciones con que trabaja.

Su impacto no es nulo: con transiciones más lentas que las caracterizadas para las celdas, los retardos calculados son extrapolaciones y por lo tanto menos precisos, y las transiciones lentas aumentan el consumo y la sensibilidad al ruido. Con el margen disponible a 100 ns no comprometen el funcionamiento, pero sí serían relevantes cerca de la frecuencia máxima. Para corregirlas habría que activar la reparación de diseño posterior al ruteo global, que en esta configuración está desactivada, o dividir las redes de alto *fanout* del banco de registros; ambas opciones modifican el circuito y obligarían a repetir el signoff.

La Figura 5 marca el banco de registros (verde), la ALU (amarillo) y el contador de ciclos (celeste). El banco de registros ocupa buena parte de la mitad izquierda, la ALU forma un bloque compacto en el centro y el contador de ciclos queda disperso en la zona inferior derecha.

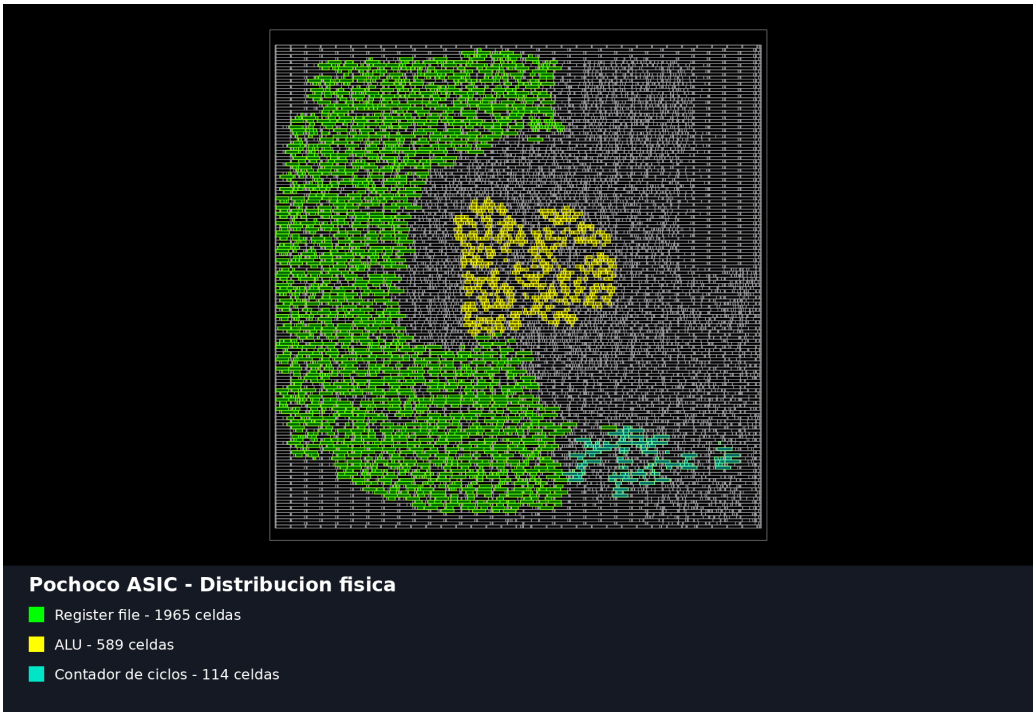


Figura 5: Layout de la corrida final con el banco de registros (verde), la ALU (amarillo) y el contador de ciclos (celeste).

Fuente: Elaboración Propia.

Las celdas de cada bloque se identificaron por el prefijo jerárquico de su nombre de instancia, que se conserva después del aplanamiento (Sección 5). El conteo se hizo sobre la netlist final (`final/nl/asic_top.nl.v`), contando las instancias de celdas de la biblioteca cuyo nombre comienza con cada prefijo (Listado 7). La Tabla 20 resume el resultado.

Listado 7: Conteo de celdas por prefijo jerárquico en la netlist final.

```
for p in "u_pochoco.u_core.u_rf." "u_pochoco.u_core.u_id.u_alu." \
         "u_pochoco.u_periph.u_cycle_counter."; do
  echo "$p $(grep -cE "^\s*gf180mcu_\S+ \\\\${p//./\\.}" final/nl/asic_top.nl.v)"
done
```

Fuente: Elaboración Propia.

Bloque	Prefijo jerárquico	Celdas	% de las celdas con prefijo
Banco de registros	`u_pochoco.u_core.u_rf.`	1.965	36,2
ALU	`u_pochoco.u_core.u_id.u_alu.`	589	10,9
Contador de ciclos	`u_pochoco.u_periph.u_cycle_counter.`	114	2,1

Tabla 20: Celdas de cada bloque en la netlist final. El porcentaje es respecto a las 5.425 celdas que conservan el prefijo `u_pochoco.`

Fuente: Elaboración Propia.

El método tiene una limitación: de las 20.503 instancias de la netlist final, 15.078 no tienen el prefijo `u_pochoco.`, porque las agregó el flujo después de la síntesis (celdas de relleno, *tap cells*, *endcaps*, *buffers* del árbol de reloj y de reparación de *timing*). Esas celdas no se atribuyen a ningún bloque, aunque algunas estén físicamente dentro de su zona. Por ejemplo, los *buffers* de *fanout* que el flujo insertó en las redes del banco de registros (Sección 8, violación de antena) no se cuentan en sus 1.965 celdas.

9. Simulación funcional post-síntesis

9.1. Configuración

Se comparó el RTL con el netlist posterior a la síntesis (`asic_top_gate.nl.v`) usando Icarus Verilog y los modelos de la biblioteca `gf180mcu_fd_sc_mcu7t5v0`, sin retardos anotados (sin SDF). Para acortar la espera de tres segundos, ambas simulaciones usan `TenthCycles = 100` en lugar de 1.000.000, de modo que 30 décimas equivalen a 3.000 ciclos. Este cambio se hizo solo en la copia del RTL de `evidencia_3_7/rtl`; el resto de esos archivos es idéntico al RTL oficial, incluido el controlador modificado de la Sección 6.4. Ambas versiones reciben el mismo reloj y los mismos estímulos en los mismos ciclos.

El netlist se generó con la síntesis de LibreLane sobre la copia del RTL de `evidencia_3_7` (que usa el mismo `config.json` del diseño), deteniendo el flujo después de la síntesis. Como el RTL y el netlist tienen el mismo nombre de módulo, el del netlist se renombró a `asic_top_gate`, para que el testbench pueda instanciar ambos a la vez. Luego se compiló todo con Icarus Verilog, usando los modelos funcionales de las celdas del PDK (`-DFUNCTIONAL`), sin retardos. El Listado 8 muestra los comandos, ejecutados en esa carpeta.

Listado 8: Generación del netlist y simulación (sección 3.7).

```
# 1. Sintesis del RTL de la simulacion (TenthCycles = 100)
librelane --pdk gf180mcuD config.json --run-tag sim37 --to Yosys.Synthesis

# 2. Renombrar el modulo del netlist
sed 's/^module asic_top(/module asic_top_gate(/' \
    runs/sim37/*-yosys-synthesis/asic_top.nl.v > asic_top_gate.nl.v

# 3. Compilar y simular (M = carpeta verilog de gf180mcu_fd_sc_mcu7t5v0 en el PDK)
iverilog -g2012 -DFUNCTIONAL -o sim_compare.vvp tb_compare.v \
    rtl/*.v rtl/espino_core/*.v rtl/mem_s1/*.v asic_top_gate.nl.v \
    $M/primitives.v $M/gf180mcu_fd_sc_mcu7t5v0.v
vvp sim_compare.vvp | tee comparacion.log
```

Fuente: Elaboración Propia.

El procedimiento se verificó repitiéndolo de forma independiente: el netlist obtenido es idéntico, byte a byte, al de `evidencia_3_7`, y la simulación reproduce los mismos ciclos que `comparacion.log`. Como el netlist proviene del RTL con `TenthCycles = 100`, no es el netlist de la corrida final; difiere de este solo en la constante del divisor de décimas de segundo. Como la simulación no tiene retardos, el período del reloj del testbench (10 ns) no influye en el resultado: lo que se compara es el comportamiento ciclo a ciclo.

9.2. Testbench

El testbench `tb_compare.v` instancia `asic_top` (RTL) y `asic_top_gate` (netlist) con las mismas entradas, y en cada ciclo compara con `!==` los LEDs, ambos displays y la salida SPI; ante cualquier diferencia, la simulación se detiene con `$fatal`. La secuencia de estímulos es la siguiente:

1. Reset durante 8 ciclos y pulsación inicial 150 ciclos después, sostenida por 50 ciclos.
2. Ronda correcta: se espera el LED objetivo, se presiona su botón 250 ciclos después y se mantiene 40 ciclos.
3. Ronda con error: se espera el LED objetivo y, 250 ciclos después, se presiona un botón distinto (el objetivo rotado un bit), sostenido por 40 ciclos.

Los ciclos medidos de una ronda se cuentan desde que se encienden los cuatro LEDs al inicio de la ronda hasta que vuelven a encenderse al inicio de la siguiente.

9.3. Resultados

Evento	**RTL (ciclo)**	**Netlist (ciclo)**	**Diferencia**
Prueba 1: ronda correcta (objetivo 0010)			
Inicio de la ronda	225	225	0
Botón correcto presionado	3.504	3.504	0
Duración de la ronda (ciclos)	3.341	3.341	**0**
Prueba 2: pulsación incorrecta (objetivo 0100, botón 1000)			
Botón incorrecto presionado	6.806	6.806	0
Display “EE” con LEDs apagados	6.827	6.827	0
Inicio de la nueva ronda	6.881	6.881	0

Tabla 21: Comparación entre la simulación RTL y la del netlist posterior a la síntesis.
Fuente: Elaboración Propia.

Listado 9: Extracto de `comparacion.log`.

```
RESULTADO RONDA CORRECTA
RTL: 3341 ciclos
GATE: 3341 ciclos
RONDA CORRECTA: OK
...
Error detectado: RTL=6827 GATE=6827
Ronda reiniciada: RTL=6881 GATE=6881
 TEST 3.7 OK
 RTL Y POSTSINTESIS COINCIDEN
```

Fuente: Elaboración Propia.

En la ronda correcta, el LED objetivo aparece unos 3.030 ciclos después del inicio (3.000 de espera más la latencia del programa) y el botón se presiona 250 ciclos después. Los 3.341 ciclos de la ronda incluyen además los 40 ciclos de pulsación y unos 20 ciclos de procesamiento antes de la ronda siguiente. Ambas versiones coinciden en estos valores y en cada ciclo de la simulación en LEDs, displays y SPI.

En la prueba con error, ambas versiones muestran “EE” con los LEDs apagados 21 ciclos después de la pulsación incorrecta y vuelven a encender los cuatro LEDs en el ciclo 6.881, es decir, reinician la ronda actual.

Como el testbench avanza esperando patrones exactos de LEDs y displays, el resultado también indica que el programa no depende del contenido inicial del banco de registros, que en el ASIC ya no arranca en cero (Sección 3). Esto es coherente con el análisis de `game.s`, en el que cada registro se escribe antes de ser leído.

9.4. Archivos entregados

En `evidencia_3_7`: el testbench `tb_compare.v`, el RTL utilizado, el netlist `asic_top_gate.nl.v`, el registro completo `comparacion.log` y el resumen `resumen_3_7.txt`.

10. Conclusión del Proyecto

Se llevó el Pochoco SoC, con el juego de reflejos del Proyecto 2, desde el RTL hasta un GDSII en GF180MCU mediante LibreLane. La corrida final, con $T = 100$ ns y $U = 40\,\%$, cumple todos los criterios de cierre: el flujo termina completo, el DRC (Magic) y el LVS están limpios, las 10 violaciones de antena detectadas se corrigieron con 12 *jumpers*, y el diseño cumple setup y hold en todos los corners, con un margen de setup de +46,89 ns en el corner lento.

El paso de FPGA a ASIC requirió reemplazar cinco construcciones que dependían de la FPGA. Una de las lecciones principales es que el flujo no siempre las advierte: la BRAM del banco de registros detuvo el flujo con un error, pero la carga de la memoria con `$readmemh` y el reset de encendido solo dejaron mensajes informativos de Yosys, y la frecuencia de reloj asumida no produjo ninguna advertencia directa. Un diseño que pasa por el flujo sin errores no es necesariamente correcto en silicio, por lo que estos supuestos deben revisarse en el propio RTL.

La decisión con mayor impacto fue la estrategia de memoria. La ROM combinacional generada por `assembler_v2.py`, junto con una RAM de datos reducida a 2 palabras, ocupó 4,2 veces menos área de celdas que la memoria unificada de Flip-Flops y permitió una frecuencia unas 1,7 veces mayor. Esta decisión se basó en datos de las propias corridas: el análisis de `game.s` mostró que el juego usa 100 palabras de instrucciones y ninguna de datos, y la síntesis mostró que la RAM de 16 palabras ocupaba cerca de un tercio del área sin ser utilizada. En el diseño final, el banco de registros es el bloque dominante, con el 48,9 % del área de celdas, mientras que el contador de ciclos agregado en el Proyecto 2 representa el 2,6 %.

El barrido de período estableció una frecuencia máxima de 18,80 MHz ($T_{\text{mín}} = 53{,}2$ ns), limitada por una ruta que, en un solo ciclo, decodifica una instrucción, calcula en la ALU la dirección de un acceso a un periférico y actualiza su registro de lectura. La modificación propuesta, que actualiza ese registro en todos los ciclos, mejoró el *slack* en 1,57 ns y redujo levemente el área. Sin embargo, la mejora fue acotada porque la siguiente ruta crítica, la resolución de saltos, era casi igual de larga. Este resultado muestra que acortar una ruta solo es útil si no existe otra de longitud similar, y que una mejora mayor requeriría cambiar la microarquitectura del *pipeline*, por ejemplo resolviendo los saltos en una etapa posterior. El intento previo sobre el controlador, en cambio, no mejoró el *timing* porque era lógicamente equivalente al original y no estaba en la ruta crítica.

En cuanto al área física, el diseño completa el flujo hasta una utilización configurada de 73 %, con un *die* de 0,228 mm^2, un 44 % menor que el de la corrida final. Con un costo lineal de US$23.000 por mm^2, la corrida final equivale a unos US$9.351, y la configuración de menor área a unos US$5.250. Esta estimación omite costos fijos, rendimiento de fabricación, *pads* y encapsulado, por lo que sirve para comparar decisiones de diseño, no para presupuestar un chip real.

La simulación del netlist posterior a la síntesis reprodujo, ciclo a ciclo, el comportamiento del RTL, tanto en una ronda correcta como en una pulsación incorrecta. Además, el procedimiento se repitió de forma independiente, obteniendo un netlist idéntico y los mismos resultados, lo que respalda la reproducibilidad del flujo.

Quedan como trabajo futuro la incorporación al macro de la optimización de la lectura de periféricos, la corrección de las violaciones de *max slew* y *max cap* mediante la reparación de diseño posterior al ruteo global, la definición de un orden de pines que agrupe cada interfaz en un solo borde, y la implementación del *bootloader* por SPI, que permitiría cambiar el programa sin volver a fabricar el chip.